

Aprendizaje Automático II - Trabajo Práctico 1

Redes neuronales feed forward y convolucionales

TUIA - FCEIA, UNR

Grupo: 5

Integrantes	Legajo
Facundo Molina	M-7128/5
Martín Facundo Lapolla	L-3372/3
Jonatan Gallardo	G-5970/6
Gabriel Soda	S-5679/1

Introducción y criterio de trabajo

El trabajo aplica redes neuronales a problemas de clasificación y regresión. Este informe resume el procedimiento, las decisiones, los desafíos y los aprendizajes. El código, los gráficos y el detalle de los experimentos se encuentran en los notebooks de cada ejercicio.

Para asegurar reproducibilidad implementamos el uso de semillas. Separamos entrenamiento, validación y test. Train ajusta los pesos, validación permite elegir las configuraciones y test se reserva para evaluar el modelo seleccionado. Los transformadores que aprenden valores de los datos se ajustan solo con train.

1. Clasificación binaria de ACV

1.1. Procedimiento y datos

Trabajamos con 5.110 pacientes. La variable objetivo es **stroke**. Hay 249 positivos (4,87 %) y 4.861 negativos (95,13 %). No se encontraron filas completas ni identificadores duplicados. Los únicos faltantes fueron 201 valores de BMI (3,93 %).

Exploramos las variables con histogramas, boxplots, correlaciones y proporciones por categoría. Edad presentó la mayor correlación de Spearman con **stroke** entre las variables examinadas (0,25). Glucosa y BMI mostraron valores extremos y solapamiento entre clases. Conservamos esos extremos para no eliminar posibles casos relevantes. Estas asociaciones no prueban causalidad.

Eliminamos **id** y dividimos de forma estratificada en 3.066 pacientes para train, 1.022 para validación y 1.022 para test. Validación y test tienen 50 positivos cada uno.

1.2. Decisiones y justificación

Completamos BMI con la mediana de train por intervalos de edad de 20 años y género. La mediana global de train funciona como respaldo. Elegimos medianas porque los valores extremos influyen menos que sobre la media. La agrupación permite incorporar diferencias entre grupos, aunque no se probó que supere otras imputaciones.

Usamos One-Hot para categorías y variables binarias, sin asignarles un orden artificial. Dejamos una categoría de referencia con `drop="first"`. Conservamos `Unknown` en tabaquismo porque es una categoría explícita. Estandarizamos edad, glucosa y BMI con `StandardScaler`. El resultado tiene 16 entradas.

El MLP de referencia tiene arquitectura 16-64-1 y 1.153 parámetros. Usa ReLU y dropout de 0,3 en la capa oculta. La salida entrega un logit y la pérdida es `BCEWithLogitsLoss`. Aplicamos sigmoid al evaluar para obtener scores entre 0 y 1.

Entrenamos con Adam, tasa 0,001, `weight_decay=0,0001` y batches de 64. Permitimos hasta 500 épocas, con paciencia 30 y mejora mínima de pérdida de validación de 0,0001. Restauramos los mejores pesos para limitar el sobreajuste.

1.3. Experimentos y resultados

Las curvas y la evaluación en test se muestran en el Anexo A.

Comparamos BorderlineSMOTE, aplicado solo a train, con la ponderación de positivos en la pérdida. BorderlineSMOTE llevó ambas clases a 2.917 registros. La ponderación mantuvo los pacientes originales y usó `pos_weight=2917/149`, aproximadamente 19,58. También comparamos tres arquitecturas con ponderación.

Arquitectura	Parámetros	F2 en validación	Recall en validación
16-64-1	1.153	0,4366	84 %
16-32-1	577	0,4343	82 %
16-64-32-1	3.201	0,4310	80 %

Elegimos 16-64-1 por su mayor F2. Las diferencias fueron pequeñas y corresponden a una sola semilla. El modelo de 32 neuronas sigue siendo una alternativa razonable por su menor tamaño.

Seleccionamos los umbrales con F2 en validación. Esta métrica da más importancia a recall que F1, porque buscamos reducir los falsos negativos. Con el umbral elegido para cada método, `pos_weight` obtuvo F2 de 0,4366 frente a 0,3779 de BorderlineSMOTE. Detectó 42 positivos frente a 39 y generó 239 falsos positivos frente a 277.

La configuración final fue el modelo ponderado, con pesos de la época 16 y umbral 0,567368. El entrenamiento se detuvo en la época 46. La pérdida de train seguía bajando cuando validación dejó de mejorar, lo que respaldó recuperar un checkpoint anterior. Las pérdidas ponderadas no se comparan directamente con las de BCE sin ponderar.

Métrica de test	Resultado	Métrica de test	Resultado
Accuracy	75,64 %	Precisión	14,34 %
Recall	80,00 %	F1	0,2432
F2	0,4175	AUC-ROC	0,8487
Especificidad	75,41 %	Precisión promedio (AP)	0,3151

La matriz de test contiene 733 verdaderos negativos, 239 falsos positivos, 10 falsos negativos y 40 verdaderos positivos. Una predicción siempre negativa tendría 95,11 % de accuracy, pero no detectaría ningún positivo. Por eso accuracy sola resulta engañosa en este problema.

1.4. Desafíos, limitaciones y aprendizajes

El principal desafío fue detectar positivos sin producir demasiadas falsas alertas. Detectamos 40 de 50 casos, pero 239 de las 279 alertas fueron incorrectas. Los 10 falsos negativos podrían quedar sin señalar si se usara el resultado para descartar una revisión. El modelo no fue validado para tomar decisiones clínicas.

Aprendimos que tratar el desbalance y elegir el umbral son decisiones distintas. F2 permitió explicitar la prioridad de recall, pero no impuso una precisión mínima aceptable. Las limitaciones principales son los pocos positivos, una sola partición y varias decisiones sobre la misma validación. BorderlineSMOTE también puede producir indicadores categóricos fraccionarios tras One-Hot. Como mejoras proponemos repetir particiones, comparar una regresión logística ponderada y probar imputaciones o sobremuestreo adecuados para datos mixtos.

2. Clasificación multiclase de dígitos

2.1. Procedimiento y datos

Usamos MNIST, con imágenes de dígitos del 0 al 9 en escala de grises y tamaño 28×28 . Leímos los cuatro archivos IDX y verificamos encabezados, cantidades, dimensiones y etiquetas. Dividimos las 60.000 imágenes originales de train en 48.000 para entrenamiento y 12.000 para validación mediante estratificación. Conservamos las 10.000 imágenes oficiales de test.

Mostramos una grilla de imágenes, las frecuencias de cada dígito y los histogramas de intensidad. Las clases de train representan entre 9,04 % y 11,24 % del total. Consideramos el conjunto aproximadamente balanceado y no aplicamos sobremuestreo. En los histogramas predominan los píxeles negros del fondo.

2.2. Decisiones y justificación

Aplanamos cada imagen en 784 valores y convertimos los píxeles a float32. Dividimos por 255 para llevarlos a $[0, 1]$. Esta transformación fija conserva el orden y las frecuencias de las intensidades. Usamos intervalos equivalentes en ambos histogramas para comparar correctamente el antes y el después.

Mantenemos las etiquetas como enteros de 0 a 9 y las convertimos a `torch.long`. CrossEntropyLoss acepta esos índices, por lo que no hace falta usar one-hot. El modelo entrega logits y no aplica softmax antes de la pérdida. Softmax se usa para analizar la confianza.

Componente	Configuración y motivo
Arquitectura	784-128-64-10, con 109.386 parámetros. Dos capas ocultas para combinar patrones con un tamaño moderado.
Activaciones	ReLU en las capas ocultas para aprender relaciones no lineales. Salida lineal con diez logits.
Regularización	Dropout de 0,3 y decaimiento de pesos de 0,0001 para limitar el sobreajuste.
Optimización	AdamW con tasa inicial de 0,001 y batches de 64.
Tasa de aprendizaje	ReduceLROnPlateau, factor 0,5 y paciencia 2, según pérdida de validación.
Selección de pesos	Hasta 30 épocas y early stopping tras 7 épocas sin nueva pérdida mínima de validación.

Al terminar cada época medimos train y validación con pesos fijos y dropout desactivado. También conservamos las métricas obtenidas durante las actualizaciones. Así distinguimos el proceso de optimización de la comparación entre conjuntos.

2.3. Entrenamiento y sobreajuste

El Anexo B reúne las curvas, la matriz y los ejemplos de errores.

La menor pérdida de validación se obtuvo en la época 18 (0,0793). El entrenamiento se detuvo en la 25 y recuperamos esos pesos. Entre las épocas 18 y 25, la pérdida de train en evaluación bajó de 0,0143 a 0,0082. La de validación osciló y terminó en 0,0834.

En la última época, train en evaluación alcanzó aproximadamente 99,8 % de accuracy frente a 98,0 % de validación. Esta diferencia muestra sobreajuste, aunque no hubo una caída sostenida

fuerte de accuracy de validación. El cruce con la curva de train durante las actualizaciones no basta para identificar su inicio, porque esa curva incluye dropout activo.

2.4. Resultados y análisis de errores

El checkpoint seleccionado obtuvo 97,94 % de accuracy y pérdida 0,0756 en test. Acertó 9.794 imágenes y falló en 206. Accuracy es un resumen adecuado por el balance aproximado de las clases, pero la complementamos con resultados por dígito y la matriz de confusión.

Dígito	Aciertos por clase	Dígito	Aciertos por clase
0	99,08 %	5	97,76 %
1	99,12 %	6	98,23 %
2	97,97 %	7	97,57 %
3	98,22 %	8	96,61 %
4	98,17 %	9	96,53 %

Los aciertos por clase se calculan sobre los ejemplos reales de cada dígito, por lo que corresponden a su recall. Las confusiones más frecuentes fueron 9 hacia 4 (13), 7 hacia 2 (8), 4 hacia 9 (7) y 7 hacia 9 (7). Agrupando ambas direcciones, 4 y 9 acumularon 20 errores. Los lazos abiertos o cerrados y los trazos diagonales ofrecen posibles explicaciones visuales.

Analizamos los cinco errores de mayor confianza, mostrados en el notebook. Sus índices de test son 3520, 1681, 2654, 4601 y 5331. Corresponden a 6 hacia 4, 3 hacia 7, 6 hacia 1, 8 hacia 4 y 1 hacia 6. Todos superaron 99,95 % de confianza softmax. Algunos tienen trazos ambiguos, pero el último no ofrece una semejanza clara con un 6. Estas observaciones no prueban qué características usó el modelo ni que las etiquetas estén mal.

2.5. Desafíos, limitaciones y aprendizajes

Los desafíos fueron reconocer distintas formas de escritura y comparar correctamente las curvas. Medir train y validación sin dropout permitió observar una brecha que era menos evidente con las métricas tomadas durante las actualizaciones.

El MLP conserva los píxeles al aplanar, pero debe aprender las relaciones entre sus posiciones. Una CNN podría aprovechar filtros locales y compartirlos en distintas partes de la imagen para detectar bordes, curvas y uniones. No se implementó una CNN en este ejercicio ni se demostró que mejore el resultado.

Aprendimos que una buena accuracy no garantiza confianza fiable. El resultado tampoco asegura rendimiento sobre fotos, fondos distintos o varios dígitos en una imagen. Como mejoras futuras proponemos comparar arquitecturas y pequeñas transformaciones solo sobre train, cuidando que no cambien la identidad del dígito.

3. Regresión de ganancias

3.1. Procedimiento y datos

La variable objetivo es **Profit** (ganancia), la cual es continua y presenta valores tanto positivos como negativos. Su distribución exhibe una asimetría positiva con un centro denso y colas pesadas. Se detectó una cantidad considerable de valores atípicos (alrededor del 19 %) que representan transacciones corporativas o descuentos, los cuales se conservaron por ser datos válidos para el negocio (ver **Imagen 6** en el Anexo C).

Para reducir la dimensionalidad y evitar sobreajustes, se descartaron atributos no predictivos como identificadores de filas y códigos postales, al igual que los nombres de productos o ciudades. Los estados con menos de 30 operaciones se agruparon en una categoría común. La fecha de orden se descompuso en año, mes y día de la semana. El año se tomó como categoría, mientras que el mes y el día recibieron codificación cíclica. Además, se midió la diferencia en días entre la creación de la orden y su envío.

3.2. Decisiones y justificación

Separamos los datos en 70 % para entrenamiento, 10 % para validación y 20 % para test. Por la presencia de valores atípicos, estandarizamos las variables numéricas con un escalado robusto, y codificamos las variables categóricas con One-Hot Encoding.

Elegimos un perceptrón multicapa (MLP) con tres capas ocultas de forma decreciente (128, 64 y 32 neuronas) y función de activación ReLU, finalizando con una salida de identidad para permitir números negativos en la ganancia. El modelo cuenta con 22.657 parámetros.

Implementamos optimización Adam con una tasa de aprendizaje de 0,001 y reducción programada según la evolución en validación. Como regularización, incorporamos Dropout de 0,2, decaimiento de pesos y parada temprana tras 25 épocas sin mejoras para evitar el overfitting.

3.3. Experimentos y resultados

Comparamos tres funciones de costo para medir el error: MSE (error cuadrático medio), MAE (error absoluto medio) y Huber Loss. Usamos la misma arquitectura y semillas en todas las pruebas. MSE resultó sensible a los valores atípicos, generando gradientes que inestabilizan el modelo. MAE penaliza de manera lineal, pero dificulta la convergencia final. Huber Loss combina los beneficios de ambas alternativas mediante un umbral definido.

Evaluados en validación, Huber y MAE convergieron de manera estable. En el caso de Huber Loss, el modelo redujo su error progresivamente junto con los escalones de reducción de la tasa de aprendizaje, sin divergir entre entrenamiento y validación (ver **Imagen 7** en el Anexo C). Al evaluar los modelos sobre el conjunto de test independiente, obtuvimos los siguientes resultados:

Función de costo	MAE (USD)	RMSE (USD)	R ²
MSE	\$25.30	\$161.75	0,7242
Huber (=1)	\$24.16	\$166.21	0,7087
MAE	\$24.48	\$189.10	0,6230

En el conjunto de test, el modelo con MSE obtuvo el menor RMSE por la propia naturaleza de su fórmula cuadrática. Por otro lado, Huber Loss logró el menor error absoluto medio y logró explicar el 71 % de la variabilidad de los datos (R²), ofreciendo un balance que no desestabiliza las predicciones frente a desviaciones grandes. Como se observa en la gráfica de residuos para este modelo (ver **Imagen 8** en el Anexo C), las predicciones son muy ajustadas para las transacciones de margen estándar, pero la red evidencia dificultades para estimar con precisión la magnitud real de los pedidos atípicos extremos, ya sean grandes pérdidas o ganancias.

Seleccionamos Huber Loss como el modelo final. Se consideró la opción más ajustada a la lógica del caso de uso, ya que preserva la precisión en la mayor parte de las transacciones operativas sin forzar las estimaciones para alcanzar a los registros extremos.

3.4. Desafíos, limitaciones y aprendizajes

Uno de los desafíos fue el aumento de dimensionalidad que produce el uso de One-Hot Encoding con las categorías del negocio. El entrenamiento del modelo requirió el ajuste manual de parámetros y carece de opciones nativas sencillas para determinar qué características aportan más al resultado.

Aprendimos que la elección de la función de costo tiene un efecto profundo sobre la convergencia y la adaptación a variables de cola pesada como las ganancias. Como mejora se plantea el uso de modelos de ensamble y árboles, los cuales manejan relaciones no lineales sin precisar transformaciones estrictas ni funciones continuas diferenciables.

4. Clasificación de razas con CNN

4.1. Procedimiento y datos

El objetivo fue clasificar imágenes de diez razas de perros. Elegimos grupos con características similares (como razas derivadas del caniche) para evaluar si el modelo lograba reconocer estas diferencias anatómicas o de textura.

Todas las imágenes se reescalaron y recortaron a 224x224 píxeles. Luego, se convirtieron a tensores y normalizaron utilizando los valores estadísticos de ImageNet. Observamos desbalance en las clases originales, liderado por la raza ‘Pug’ con 148 registros (ver **Imagen 9** en el Anexo D).

Para estandarizar la evaluación de las distintas arquitecturas, definimos una única función de entrenamiento parametrizada. Esta función unifica la alternancia entre optimización y evaluación, calcula las métricas, ajusta la tasa de aprendizaje, frena el entrenamiento de manera temprana y restaura los mejores pesos de forma automática.

4.2. Evolución de arquitecturas

Probamos distintas arquitecturas convolucionales:

E1 (Línea base): Red sencilla de tres bloques de convolución y agrupamiento. Logró 85 % de precisión en validación y 67 % en test, mostrando pérdida de generalización por no contar con métodos de regularización.

E2 (Profunda): Aumentó a seis capas de convolución en cuatro etapas de extracción. Requirió más procesamiento y arrojó un 68 % de precisión en test, indicando que incrementar capas sin regularización sobre conjuntos reducidos de imágenes no se traduce directamente en mejoras.

E3 (Avanzada): Integró módulos Inception para percibir detalles multiescala y conexiones residuales para favorecer el aprendizaje del gradiente. El incremento de parámetros sin regularización provocó un sobreajuste evidente y el resultado en test descendió a 62 %.

E4 (Regularizada): Mantuvo la estructura de E3 pero resolvió su fuerte sobreajuste implementando aumento de datos (data augmentation) para equilibrar las clases y aportar diversidad visual, junto con regularización por Dropout y decaimiento de pesos.

4.3. Experimentos y resultados finales

Experimento	Acc. Train	Acc. Val	Acc. Test	Parámetros	Épocas
E1 (Línea base)	99.00 %	85.00 %	67.00 %	~580 k	34
E2 (Profunda)	97.00 %	79.00 %	68.00 %	~1.62 M	25
E3 (Avanzada)	97.00 %	77.00 %	62.00 %	~2.31 M	14
E4 (Regularizada)	96.00 %	83.00 %	76.00 %	~2.31 M	53

La evolución desde el primer modelo hasta el último evidenció un problema persistente: la memorización de los datos. E1 sufrió un sobreajuste temprano por falta de regularización. Al intentar mejorar el desempeño, la profundidad lineal sin control en E2 no logró incrementar la generalización, mientras que la alta capacidad de E3 provocó una caída de aciertos en test. Incrementar de manera directa la cantidad de parámetros sobre un conjunto de entrenamiento pequeño provocó que las redes más profundas se sobreajustaran rápidamente en lugar de aprender características generales.

Para revertir esta situación, E4 no modificó la arquitectura avanzada de E3, sino que se concentró en los datos y en el control del aprendizaje. Con la implementación del balanceo de clases mediante copias transformadas (recortes variables, giros horizontales, leves rotaciones y ajustes de color), el modelo E4 mitigó la memorización. A esto se sumaron las capas de Dropout y el decaimiento de pesos, permitiendo finalmente aprovechar la capacidad de los módulos de la red y obteniendo un 76 % de precisión en las imágenes del conjunto de test. Las curvas de pérdida y precisión generadas (ver **Imagen 10** en el Anexo D) validan gráficamente que el modelo convergió de manera estable sin divergencias de sobreajuste.

La matriz de test reportó 76 aciertos (ver **Imagen 11** en el Anexo D). Las confusiones principales ocurrieron entre razas de rasgos afines y colores similares (como perros negros) y texturas de pelaje equivalentes (como los caniches). El modelo logró identificar las razas cuando las imágenes presentaban encuadres frontales y buena iluminación (ver casos de éxito y error en las **Imágenes 12 y 13** del Anexo D).

4.4. Desafíos, limitaciones y aprendizajes

El trabajo evidenció que emplear arquitecturas de alta capacidad descriptiva frente a pocos ejemplos conlleva un riesgo directo de memorización. Aprendimos que herramientas estructurales complejas como los módulos Inception y los bloques residuales logran desplegar su potencial únicamente si se acompañan de técnicas de aumento artificial de datos y regularización de la red para garantizar el aprendizaje de características generalizables.

4.5. Propuesta de mejora

Como técnica complementaria para optimizar el entrenamiento, se propone incorporar la inicialización de pesos de Kaiming. Dado que la arquitectura emplea funciones de activación ReLU que anulan las señales negativas, este método ajusta la escala inicial de los pesos basándose en la cantidad de conexiones de cada capa. Esto previene que las señales se diluyan durante el aprendizaje, propiciando un inicio de entrenamiento mucho más rápido y estable.

Anexo A. Imágenes del ejercicio 1

Gráficos extraídos de las salidas guardadas de AA2_tp1_problema1.ipynb.

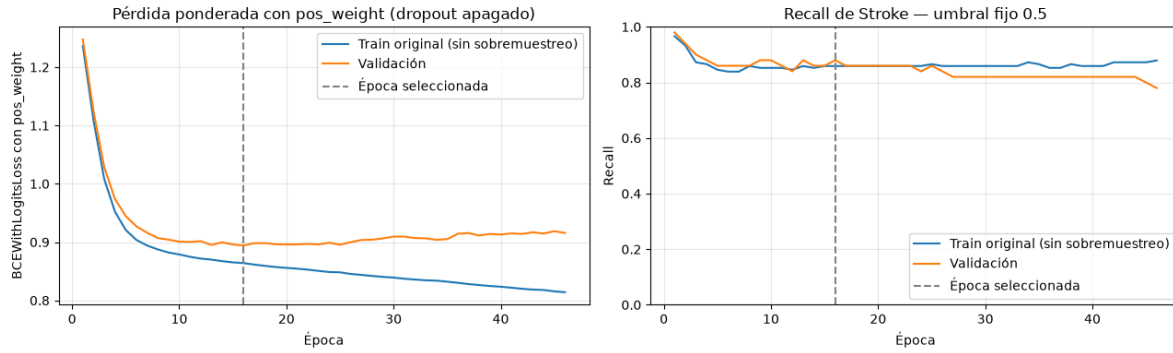


Figure 1: Imagen 1. Modelo con pos_weight. La línea marca los pesos elegidos en la época 16. Train y validación se miden sin dropout. El recall de estas curvas usa umbral 0,5 y no el umbral final elegido por F2.

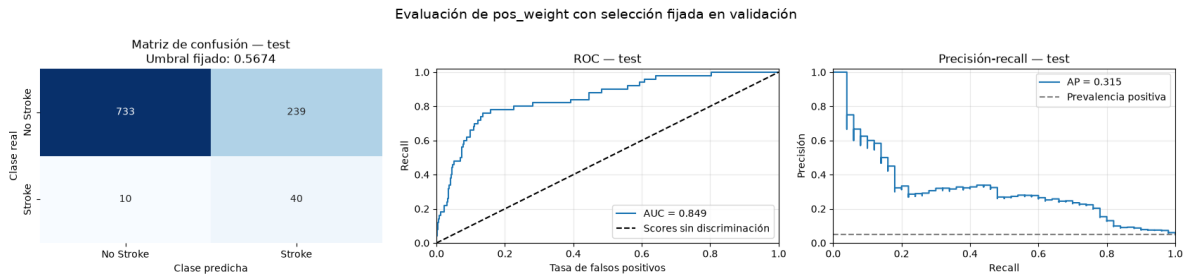


Figure 2: Imagen 2. Evaluación en test con el umbral fijado en validación. La matriz muestra 40 positivos detectados, 10 omitidos y 239 falsas alertas. Las curvas ROC y precisión-recall resumen los scores sin elegir un nuevo umbral con test.

Anexo B. Imágenes del ejercicio 2

Imágenes extraídas de las salidas guardadas de AA2_tp1_problema2.ipynb.

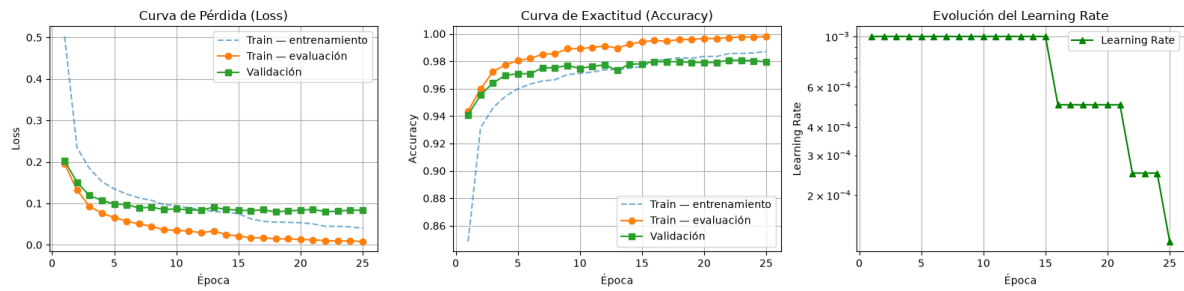


Figure 3: Imagen 3. Curvas de pérdida, accuracy y tasa de aprendizaje. Train en evaluación y validación usan los mismos pesos al terminar cada época y no tienen dropout activo. La línea discontinua registra train durante las actualizaciones.

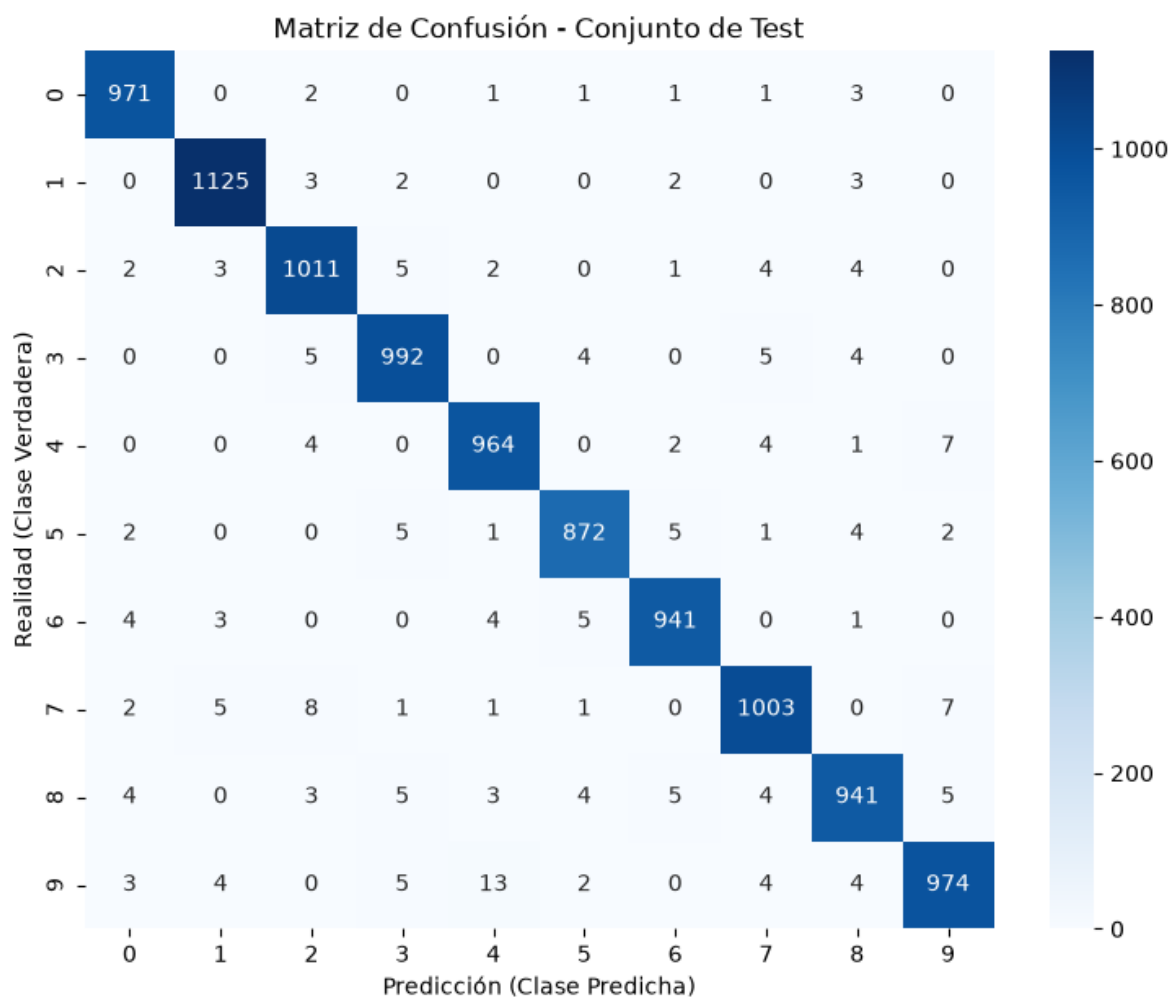


Figure 4: Imagen 4. Matriz de confusión de test. Las filas indican el dígito real y las columnas la predicción. Las confusiones entre 4 y 9 suman 20 errores.



Figure 5: Imagen 5. Cinco errores con mayor confianza softmax. Los índices permiten identificar las imágenes. Todos superan 99,95 % de confianza, aunque sus predicciones no coinciden con las etiquetas del dataset.

Anexo C. Imágenes del ejercicio 3

Gráficos extraídos de las salidas guardadas de AA2_TP1_E3.ipynb.

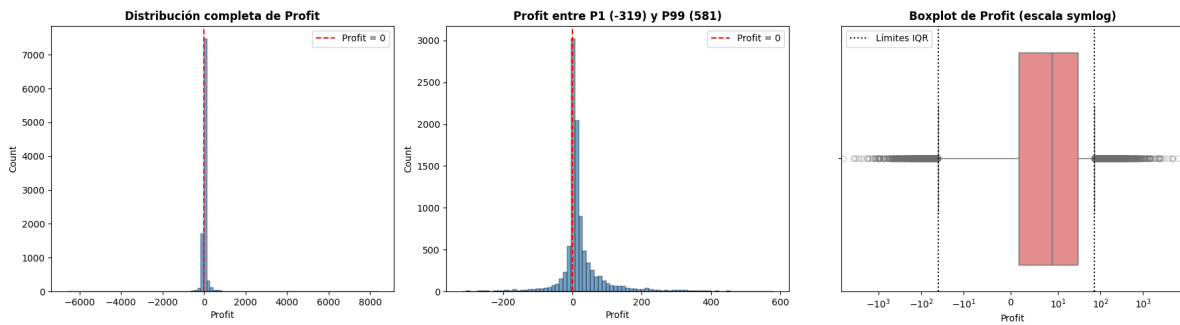


Figure 6: Imagen 6. Distribución de la ganancia (Profit). Muestra la concentración en valores cercanos a cero junto con las pesadas colas producidas por los valores atípicos corporativos o de grandes descuentos.

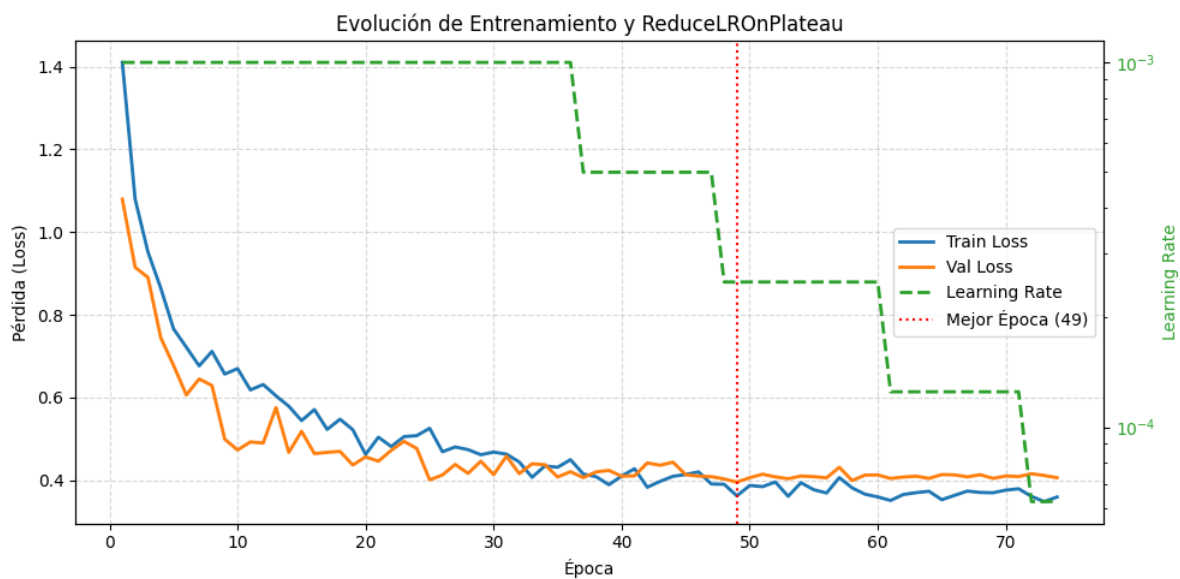


Figure 7: Imagen 7. Curvas de pérdida de Huber Loss y evolución del Learning Rate. Se observa una convergencia estable y sostenida a lo largo de las épocas, sin evidencia de sobreajuste entre entrenamiento y validación.

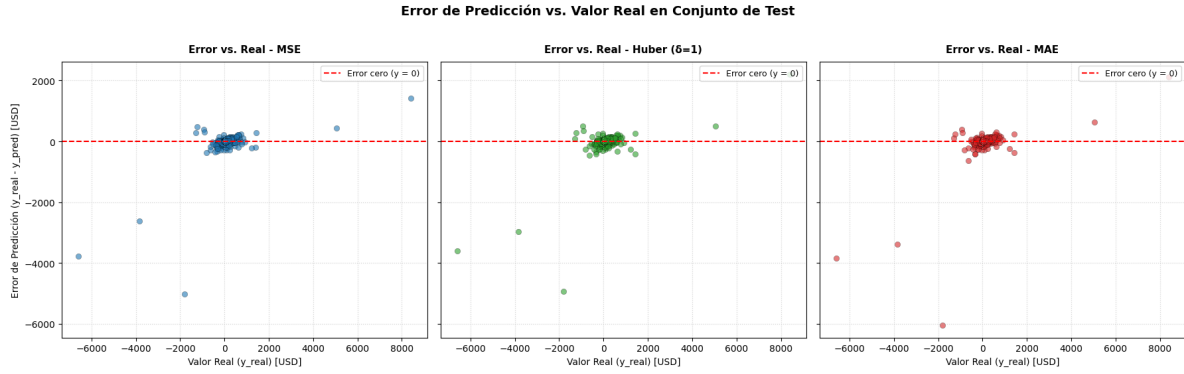


Figure 8: Imagen 8. Gráfico de dispersión Residuos vs. Predicho (Huber Loss). Exhibe cómo el modelo ajusta de forma precisa en las transacciones comunes, pero subestima o sobreestima la magnitud de las operaciones atípicas extremas.

Anexo D. Imágenes del ejercicio 4

Gráficos extraídos de las salidas guardadas de AA2_TP1_E4.ipynb.

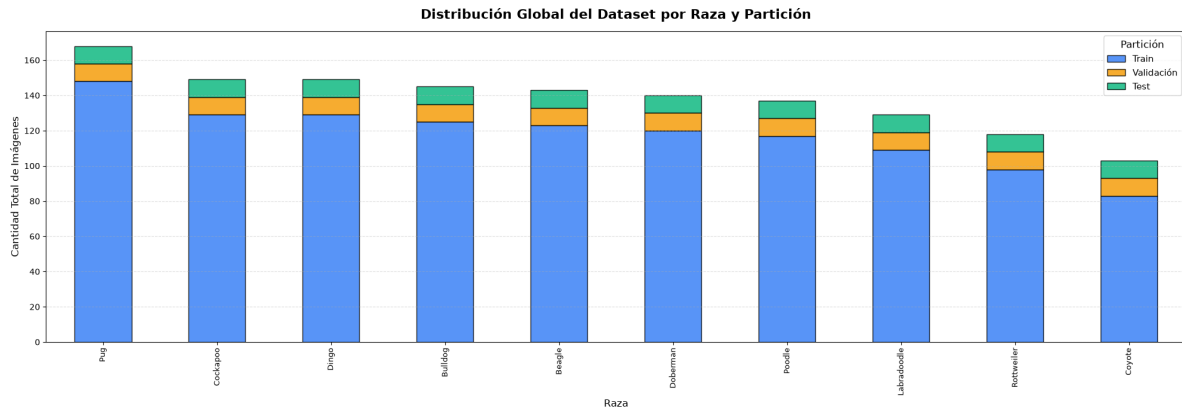


Figure 9: Imagen 9. Distribución de clases original (EDA). Muestra el evidente desbalance en la cantidad de observaciones por raza, justificando el uso de técnicas de aumento de datos en el experimento E4.

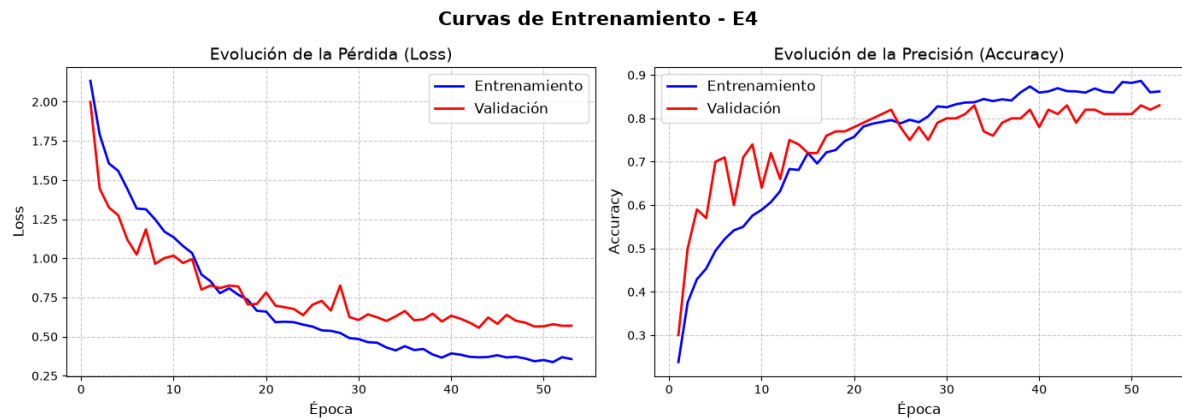


Figure 10: Imagen 10. Curvas de pérdida y precisión del modelo E4. Muestra una convergencia sostenida gracias a la regularización, marcando con una línea punteada el punto óptimo antes del sobreajuste en la época 43.

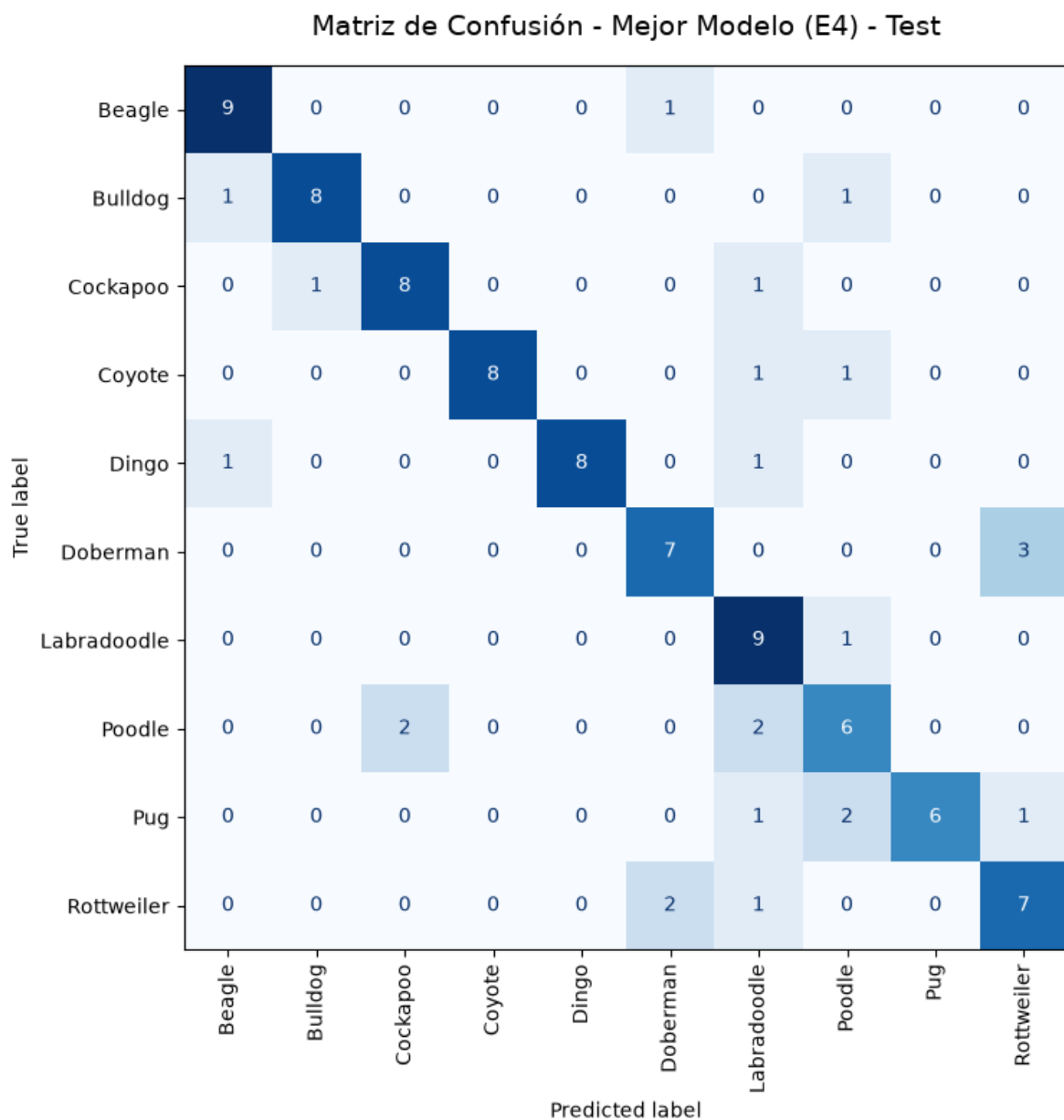


Figure 11: Imagen 11. Matriz de confusión de test (E4). Expone el rendimiento detallado por clase, donde destacan las confusiones por textura de pelaje (Poodle vs. Cockapoo) y por color (Doberman vs. Rottweiler).



Figure 12: Imagen 12. Ejemplos de clasificaciones correctas (E4). El modelo identifica con precisión a las razas cuando las imágenes presentan buena iluminación, contraste adecuado y encuadres representativos del animal.



Figure 13: Imagen 13. Ejemplos de clasificaciones incorrectas (E4). Los errores ocurren debido al solapamiento morfológico real (como pelajes rizados similares) o por encuadres inusuales que ocultan la estructura anatómica.